
Deep Reinforcement Learning for Autonomous Driving on Multilane Highway

Lazzari Tommaso

Abstract

This report investigates the application of Deep Reinforcement Learning (DRL) to autonomous driving on a multilane highway. Using the `HighwayEnv` simulation suite, I design and evaluate a cautious deterministic baseline policy as a benchmark, and compare it against a self-learning agent trained via Proximal Policy Optimization (PPO). Furthermore, I analyze the impact of reward shaping by introducing a crash penalty (malus) to prioritize vehicle survival over speed and lateral alignment. Finally, I explore the scalability of both configurations across highway environments with 3, 4, and 5 lanes. My results show that while the heuristic baseline is highly competitive in narrow environments, PPO with crash malus achieves a 0.0% collision rate during evaluation across all lane configurations, demonstrating the efficacy of safety-regularized RL policies in complex spatial environments.

1. Introduction

In this project's task, the ego-vehicle is driving on a multilane highway populated with other vehicles. The agent's objective is to reach a high speed while avoiding collisions with neighbouring vehicles. Driving on the right side of the road is also rewarded. The conducted experiment first consisted of developing a baseline policy, an educated guess on how to solve the problem. The developed baseline policy is a heuristic deterministic policy, which emulates a cautious driver decision making. This baseline policy was then evaluated on a 10-episode test run and the results were used as a benchmark for the subsequent parts of the experiment. Secondly, an agent was trained using a Proximal Policy Optimization (PPO) algorithm supported by a two-neural-network actor-critic structure. This agent was trained twice: first using the default reward function (Eq. 2) and then using a modified reward function incorporating a larger crash penalty (Eq. 3). Lastly, to verify the consistency of the previous observations, the same experiment was conducted on slightly different environments: specifically, on 4-lane and 5-lane highway environments.

2. Environment

The project uses the `HighwayEnv` library (Leurent, 2018), which is specified by a state space, a discrete action space and a reward function. The state space consists of a $V \times F$ array that describes a list of $V = 5$ vehicles by a set of features of size $F = 5$. The features for each vehicle are:

- presence (boolean), presence
- normalized position along the x axis w.r.t. the ego vehicle, x
- normalized position along the y axis w.r.t. the ego vehicle, y
- normalized velocity along the x axis w.r.t. the ego vehicle, v_x
- normalized velocity along the y axis w.r.t. the ego vehicle, v_y

$$[\text{presence}, x, y, v_x, v_y]$$

The discrete action space contains 5 possible actions: change lane to the left, `LANE_LEFT`; idle, `IDLE`; change lane to the right, `LANE_RIGHT`; go faster, `FASTER`; go slower, `SLOWER`.

2.1. State Representation and Flattening

To process the environmental observation with standard fully-connected feed-forward neural networks, the $V \times F = 5 \times 5$ observation matrix must be reshaped. Since fully-connected layers expect a one-dimensional feature vector, the observation matrix is flattened into a 25-dimensional state vector $s \in \mathbb{R}^{25}$:

$$s = [\text{presence}_1, x_1, y_1, v_{x,1}, v_{y,1}, \dots, \text{presence}_5, x_5, y_5, v_{x,5}, v_{y,5}]^T \quad (1)$$

where index 1 denotes the ego-vehicle itself, and indices 2 through 5 represent the neighboring vehicles. This vector representation preserves all numerical values while enabling the neural network to learn relationships across vehicles and their relative positions.

2.2. Reward Function

The default reward function $R(s, a)$ in the highway-fast-v0 environment is designed to balance speed, traffic rules, and safety. It is defined as:

$$R(s, a) = w_v \left(\frac{v - v_{\min}}{v_{\max} - v_{\min}} \right) + w_l d_{\text{right}} - w_c \mathbb{I}(\text{collision}) \quad (2)$$

where v is the ego-vehicle's current longitudinal velocity, $v_{\min}$ and $v_{\max}$ represent the velocity limits of the environment (20 m/s and 30 m/s by default), and d_{right} is a lane-dependent factor reward ($d_{\text{right}} = 1$ if the ego-vehicle is in the rightmost lane, and decreases linearly towards the left lanes). The term $\mathbb{I}(\text{collision})$ is an indicator function that equals 1 if a collision occurs and 0 otherwise. The default weights are configured as:

- $w_v = 0.4$ (velocity reward weight)
- $w_l = 0.1$ (rightmost lane compliance weight)
- $w_c = 1.0$ (standard collision penalty weight)

To prioritize safety and survival over speed, a modified reward function incorporating a *crash malus* was introduced. This is implemented by adding an extra penalty of -5.0 at the terminal step when the episode terminates due to a collision:

$$R_{\text{modified}}(s, a) = R(s, a) - 5.0 \cdot \mathbb{I}(\text{crash}) \quad (3)$$

3. Critical Issues

Before detailing the design of the baseline and the PPO agent, it is worth outlining the main challenges encountered:

- **Relative vs Absolute Positioning:** As described in Section 2, the observation vector s does not report the absolute position of surrounding vehicles, but only their position (x, y) normalized with respect to the ego-vehicle. This is particularly critical for the design of a deterministic, rule-based baseline (Section 4), since lane-changing decisions inherently require knowing which lane a given vehicle currently occupies.
- **Generalization Across Environments:** A further challenge was designing a training and evaluation pipeline general enough to operate on highway environments with a different number of lanes (3, 4, and 5), as required by the scalability analysis in Section 6.3.
- **Computational Resources:** Finally, the available computational budget constrained several aspects of the experimental design. The size of the Actor and Critic

Algorithm 1 Baseline

```

if a front vehicle is too close then
  if LANE_RIGHT is available and right lane is safe
  then
    action  $\leftarrow$  LANE_RIGHT
  else if LANE_LEFT is available and left lane is safe
  then
    action  $\leftarrow$  LANE_LEFT
  else
    action  $\leftarrow$  SLOWER
  end if
else if
  if FASTER is available then
    action  $\leftarrow$  FASTER
  else
    action  $\leftarrow$  IDLE
  end if
end if
    
```

networks (Section 5.1) was deliberately kept at two hidden layers of 256 units each, balancing representational capacity against training time. Similarly, the total number of experiments had to be limited, since each additional configuration required a full, independent PPO training run of 60,000 steps to obtain a reliably converged policy.

4. Baseline

As a baseline policy, a deterministic, non-learning policy was implemented. The determinism drastically reduced the computational costs of the project compared to the use of a trained policy and made the decision-making process of the algorithm extremely easy to understand. In fact, the policy was built emulating the choice of a human safe driver.

As shown in Algorithm 1, the agent following the baseline policy follows simple driving rules: keep moving as fast as possible when the road ahead is clear; when a vehicle is too close in front, move to the right lane (as a first option, because it provides more points under Eq. 2) or to the left lane if they are safe; if neither lane is safe, slow down.

This decision-making process is supported by four hyperparameters in particular. These hyperparameters' values were determined through a simple grid search.

- CAUTION_FRONT = 0.3 determining whether the vehicle ahead is too close and some safety manoeuvre is necessary
- SAFE_FRONT = 0.3 determining whether there would be enough free space in front of the ego vehicle in a given target lane.

- `SAFE_REAR` = 0.1 determining whether there would be enough free space behind the ego vehicle in a given target lane. Combined with `SAFE_FRONT`, it determines whether a lane the agent wants to move to is safe or not. Therefore, a target lane is considered unsafe if there exists a present vehicle (`presence` = 1) in that lane such that: $0 < x < \text{SAFE_FRONT}$ or $-\text{SAFE_REAR} < x < 0$.
- `LANE_THRESHOLD` = 0.25 is used to estimate lane memberships for other vehicles. The observation matrix does not provide the absolute position of other vehicles but their positions relative to the ego vehicle. Therefore, this method was found to be a simple and effective heuristic for approximating which lane each vehicle is driving in. Given the lateral position, y , of a given vehicle:
 - if $|y| \leq \text{LANE_THRESHOLD}$, the vehicle is assumed to be in the same lane as the ego vehicle
 - if $y < -\text{LANE_THRESHOLD}$, the vehicle is assumed to be in its left lane
 - if $y > \text{LANE_THRESHOLD}$, the vehicle is assumed to be in its right lane

5. PPO Agent

Given the nature of the environment, to avoid unrealistic assumptions or too complex manual feature engineering, the approach relied on a Deep Reinforcement Learning method. The Proximal Policy Optimization (PPO) (Schulman et al., 2017) algorithm was used to train the agent.

Considering the great data sampling process required to train such agent with respect to the available computational resources, PPO was deliberately chosen to avoid unnecessary costs. In fact, Advantage Actor-Critic-like methods, which force the discarding of sampled trajectories, would have been highly inefficient; on the other hand, Trust Region Policy Optimization methods would have avoided this problem by constraining the updates to prevent instability. This second method could have been very robust for this project, but enforcing the constraint requires approximations involving the Fisher information matrix and conjugate-gradient methods, making it computationally unaffordable. PPO is a simpler alternative starting from the same problem: to reuse data for more than one update, avoiding updates that move the new policy too far from the previous one. The structure relies on two neural networks: a parameterized policy, π_θ , which determines which actions among the available is to be taken; and a parameterized value function, V_ϕ , which estimates how much future rewards the current state is expected to produce.

At each step of the algorithm, the agent observes the current state, s_t , the policy chooses an action, and the environment

Algorithm 2 Proximal Policy Optimization (PPO)

Initialize policy parameters θ_0 and value function parameters ϕ_0

for $k = 0, 1, 2, \dots$ **do**

Collect a set of trajectories $D_k = \{\tau_i\}$ by running policy π_{θ_k}

Compute rewards-to-go $\hat{R}_t$

Compute advantage estimates $\hat{A}_t$ based on the current value function V_{ϕ_k}

Update the policy by maximizing the PPO-Clip objective:

$$\theta_{k+1} = \arg \max_{\theta} \frac{1}{|D_k|T} \sum_{\tau \in D_k} \sum_{t=0}^T \min \left(r_t(\theta) \hat{A}_t, \text{clip} \left(r_t(\theta), 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t \right)$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)}$

Fit the value function by minimizing the mean squared error:

$$\phi_{k+1} = \arg \min_{\phi} \frac{1}{|D_k|T} \sum_{\tau \in D_k} \sum_{t=0}^T \left(V_\phi(s_t) - \hat{R}_t \right)^2$$

end for

returns a reward, R_t , and a new state, s_{t+1} . After that, PPO computes the advantage, A_t , which is the difference between what was expected to happen and what actually happened. If the advantage is positive, the action was better than expected. The policy should therefore become more likely to choose actions with positive advantages. PPO performs this update by preventing overly large policy changes through a clipped objective function.

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t [\min(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t)]$$

where ϵ is a small positive value and $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$.

5.1. Actor-Critic Architecture and Hyperparameters

The PPO agent utilizes an Actor-Critic architecture consisting of two separate neural networks:

- **Actor Network (Policy, $\pi_\theta(a|s)$):** Estimates the probability distribution over actions for a given state. It consists of two hidden layers of 256 units each, with Rectified Linear Unit (ReLU) activations. The output layer is a linear layer mapping to 5 logits (represent-

ing the discrete actions), followed by a Categorical distribution to sample actions.

- **Critic Network (Value Function, $V_\phi(s)$):** Estimates the expected cumulative future reward from the current state. It shares the same layer structure as the Actor (two hidden layers of 256 units and ReLU activations), but outputs a single scalar value via a linear layer.

Table 1. PPO Agent Training Hyperparameters

Hyperparameter	Value
Total Training Steps	60,000
Rollout Buffer Size (T)	2,048
Minibatch Size	64
PPO Update Epochs	10
Discount Factor (γ)	0.99
GAE Lambda (λ)	0.95
Clip Parameter (ϵ)	0.20
Learning Rate (α)	3×10^{-4} (linear to 5×10^{-5})
Value Loss Coeff. (c_1)	0.50
Entropy Coeff. (c_2)	0.01
Optimizer	Adam
Maximum Gradient Norm	0.5

To encourage exploration in early training stages, the weights of the Actor’s output layer are initialized orthogonally with a very small standard deviation of 0.01, while hidden layer weights are initialized with standard deviation $\sqrt{2}$. The training hyperparameters used in the experiment are summarized in Table 1.

6. Results and Discussion

The performance of the heuristic baseline and the trained PPO agents was evaluated across different highway configurations. Table 2 provides a comprehensive quantitative comparison. It lists the ****Training Convergence**** (computed as the average of the last 100 training episodes, representing stochastic exploration policy performance) and the ****Evaluation Performance**** (computed as the average of 10 deterministic test runs using argmax action selection).

6.1. Heuristic Baseline vs. PPO Learning

In the 3-lane scenario, the deterministic hand-crafted baseline policy proves highly effective, achieving a mean return of 27.358 and a 10.0% crash rate. As shown in Fig 1, during the initial phase of training (first 100 episodes), the PPO agents operate randomly, crashing in nearly 100% of the episodes and accumulating very low returns (averaging 7.953). However, over the course of the 60,000 steps, PPO successfully learns to drive, lane-change, and avoid vehicles. In the evaluation phase (deterministic testing), the default

PPO agent achieves an average return of 26.434, and the crash-malus PPO agent reaches 28.294. The Reinforcement Learning approach was capable of developing a safe and efficient policy from scratch, exceeding my carefully structured baseline policy.

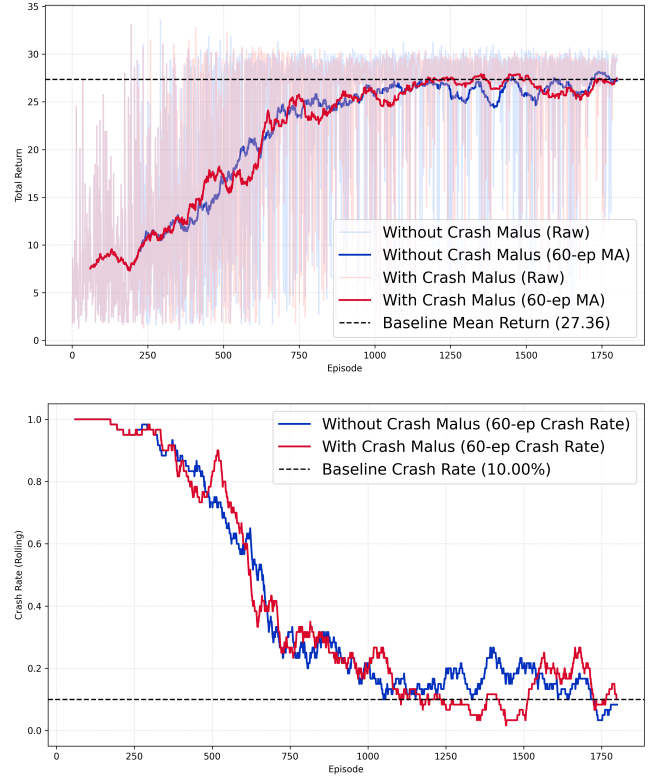


Figure 1. Training returns (top) and rolling crash rates (bottom) for both experiment with default (blue) and modified (red) reward function in the 3-lane environment. The baseline policy results are shown as a dashed line for comparison.

6.2. Impact of the Crash Malus

The introduction of the -5.0 crash malus has a significant impact on policy safety and performance:

- **Safety Improvements during Training:** The crash malus agent achieves lower training crash rates across all different environments (e.g., 9% vs. 12% in 3 lanes, and 12% vs. 15% in 4 lanes).
- **Safety in Evaluation:** During evaluation, the crash-malus agent successfully achieves a 0.0% crash rate in all scenarios (3, 4, and 5 lanes). In contrast, the default PPO agent in 3 lanes still suffers a 10% crash rate in deterministic evaluation.
- **Trade-off in Constrained Space:** In the narrow 3-

Table 2. Performance comparison of the Baseline and PPO agents across different highway lane configurations.

LANES	CONFIGURATION	TRAINING (LAST 100 EPISODES)		EVALUATION (10 EPISODES)	
		MEAN RETURN	CRASH RATE	MEAN RETURN	CRASH RATE
3 LANES	SAFETY-FIRST BASELINE	—	—	27.358	10.0%
	PPO (DEFAULT REWARD)	26.971	12.0%	26.434	10.0%
	PPO (CRASH MALUS)	26.483	9.0%	28.627	0.0%
4 LANES	PPO (DEFAULT REWARD)	26.659	15.0%	27.939	0.0%
	PPO (CRASH MALUS)	27.208	12.0%	28.294	0.0%
5 LANES	PPO (DEFAULT REWARD)	27.013	10.0%	28.028	0.0%
	PPO (CRASH MALUS)	26.968	9.0%	28.028	0.0%

lane environment, safety comes at a slight cost of efficiency during training (26.483 training return for crash malus vs. 26.971 for default). This is due to the agent adopting overly cautious behaviors (such as slowing down to avoid close vehicles instead of overtaking). However, when evaluated deterministically, the crash-malus agent achieves a superior return (28.627 vs. 26.434), showing that safety regularization prevents catastrophic failures, which drastically lower average returns.

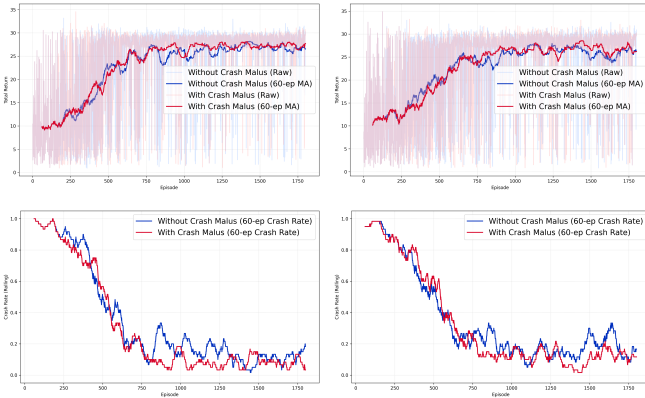


Figure 2. Training returns (top) and rolling crash rates (bottom) for both experiment with default (blue) and modified (red) reward function in the 4 (left) and 5-lane (right) environments

6.3. Scalability to Wider Highways

As the highway is widened to 4 and 5 lanes, we observe clear scaling patterns:

- **Space as a Safety Buffer:** In wider environments, even random policies crash less (crashes drop from 100% to 96% in the first 100 episodes), as the physical density of traffic decreases.

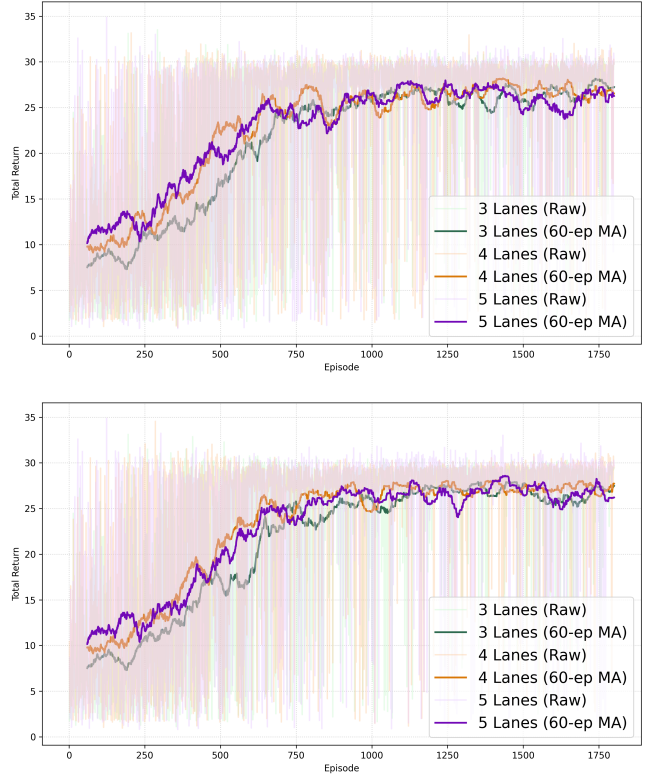


Figure 3. Training returns for agent trained with normal (top) and modified (bottom) reward function for 3, 4, and 5 lanes environment.

- **Synergy between Safety and Performance:** In the 4-lane scenario, the crash-malus agent achieves a higher training return (27.208) than the default agent (26.659) while reducing the crash rate (12% vs. 15%). In wider environments, the presence of additional lanes gives the agent physical space to bypass traffic safely. The safety-first policy keeps the agent alive longer, allowing it to collect velocity rewards over longer episode lengths.
- **Convergence Rates:** Fig 3 indicates that the crash

malus accelerates convergence in wider environments. In 4 and 5 lanes, the crash malus agent reaches a high-performance plateau (> 26.9 return) by the middle of training (episode 900), whereas it converges much slower in the highly constrained 3-lane environment due to the severe penalties associated with narrow, unavoidable traffic bottlenecks.

6.4. Stability and Variance Analysis of Evaluation Returns

To evaluate the robustness and consistency of the learned driving policies, I analyzed the variance of the total returns obtained during the 10 evaluation runs. Return variance serves as a proxy for policy stability; a high variance indicates unpredictable behaviors or failure modes (e.g., collisions), whereas a low variance demonstrates stable, risk-averse execution. The results are summarized in Table 3.

Table 3. Comparison of Mean Return, and Return Variance across 10 evaluation runs for all configurations.

Lanes	Policy Setup	Mean Return	Sample Variance
3	Heuristic Baseline	27.4	40.9
3	PPO (Default)	26.4	17.9
3	PPO (Crash Malus)	28.6	1.5
4	PPO (Default)	27.9	1.4
4	PPO (Crash Malus)	28.3	0.5
5	PPO (Default)	28.0	1.1
5	PPO (Crash Malus)	28.0	1.2

The heuristic baseline exhibits the highest variance ($s^2 \approx 40.9$). Despite achieving a solid mean return under successful runs, its rule-based design lacks the adaptability to recover from critical safety bottlenecks, resulting in a collision.

A similar variance pattern is observed for the PPO agent in the 3-lane environment. Without safety regularization, the default PPO agent suffers a collision, resulting in a high return variance ($s^2 \approx 17.9$). However, introducing the crash malus regularizer dramatically reduces the variance to $s^2 \approx 1.5$ and eliminates collisions entirely. As visually supported by the boxplot distributions (Fig 4), the default policy is spread out with a low-return outlier, whereas the crash malus policy remains tightly clustered around high-return values.

In the 4-lane environment, the PPO agent with crash malus achieves the lowest variance across all experiments ($s^2 \approx 0.5$). This indicates that the combination of spatial freedom (an additional lane) and safety penalty allows the policy to settle on highly consistent, crash-free trajectories.

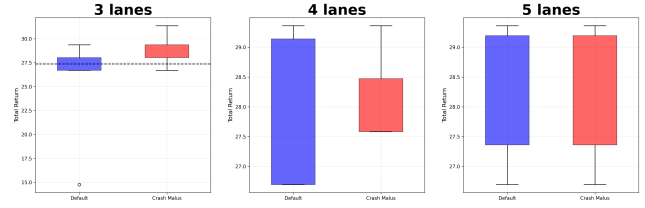


Figure 4. Training returns' distribution for all configurations

Interestingly, in the 5-lane highway environment, the variances of the default and crash malus agents converge to almost identical values ($s^2 \approx 1.1$ and $s^2 \approx 1.2$, respectively), with both configurations achieving a 0.0% collision rate. This suggests that in wider environments, the natural decrease in traffic density mitigates spatial constraints. Consequently, the default PPO agent can safely navigate without collisions, diminishing the regularizing effect of the crash malus on return stability. Another hypothesis is that the two 256-unit hidden layers that define the agent are large enough to improve the policy as long as the number of possible environment configurations remains limited. As the size of the environment increases, however, the capacity of the hidden layers might become a limiting factor in the agent's learning process.

7. Conclusions

The results support a clear conclusion: while the default-reward PPO agent, without safety regularization, fails to surpass the baseline (26.434 vs. 27.358 return), the PPO agent trained with the crash malus consistently and robustly outperforms the heuristic baseline across every metric considered. It achieves a higher evaluation return (28.627 vs. 27.358), a better crash rate (0.0% vs. 10.0%), and, crucially, a return variance more than 25 times lower (1.5 vs. 40.9).

Several directions could further strengthen the results and address the limitations discussed in Section 3. Among them, in my opinion, the most promising would be:

- **Deeper/wider networks:** larger architectures, or alternative structures could better capture inter-vehicle relationships and potentially improve generalization to more crowded or wider environments.
- **Alternative reward shaping:** Beyond the crash malus explored here, it would be valuable to test different reward functions. One option could be to increase the speed bonus weight to study the safety-speed trade-off more finely.

References

- Leurent, E. An environment for autonomous driving on highway. <https://github.com/eleurent/highway-env>, 2018.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.